FastSpeech 2 Text-to-Speech for English and Telugu

Technical report · Akshaya Bhargavi Pyla · November 2026
Code, results and checkpoints: see the GitHub repository linked in the README [CONFIRM: add repo URL]

1. What I did, in short

The task was to train FastSpeech 2 twice, once on English and once on Telugu, from about 5 hours of training audio each, and then look closely at what the duration, pitch and energy predictors do on a fixed set of test sentences. I started from the reference FastSpeech 2 implementation that the assignment points to [CONFIRM: ming024/FastSpeech2] and kept its model as it is: a transformer encoder, a variance adaptor with three predictors (duration, pitch, energy) and a length regulator, then a mel decoder with a postnet. The model is trained on five loss terms: mel before the postnet, mel after the postnet, pitch, energy and duration. I did not change the architecture. What I changed was everything around it, because that is where the two languages differ.

Most of my time went into the data side. FastSpeech 2 needs a phone-level alignment for every training utterance, so the first thing I had to get right was text → phones → alignment, and for Telugu that front end did not exist. The English model reuses the ARPAbet phone set and the pretrained Montreal Forced Aligner (MFA) model. The Telugu model needed a grapheme-to-phone mapper that I wrote, a pronunciation dictionary built from it, and an aligner that I trained from scratch on the Telugu corpus itself.

Changes compared with the reference code

Part	English	Telugu
Text front end	ARPAbet phones from the MFA `english_us_arpa` dictionary. Numbers are spelled out before alignment. Words missing from the dictionary get pronunciations from MFA's G2P model for training and from `g2p_en` at inference.	New rule-based Telugu grapheme-to-phone mapper (`telugu_prep.py`) with a 47-phone inventory: short and long vowels, aspirated and retroflex stops, geminates, and a nasal that assimilates to the following consonant. Numerals are expanded into Telugu words at inference.
Alignment	Pretrained MFA acoustic model `english_us_arpa`.	MFA acoustic model trained from scratch on the 2,762 selected utterances (monophone, triphone, LDA, three SAT passes; about 2 h 18 min).
Speakers	Multi-speaker (115 speaker ids, see section 2).	Single speaker.
Training loop	`--resume` support, validation every 1,000 steps on the full validation set, loss logged to CSV every 100 steps, best checkpoint chosen by validation loss, and checkpoints/logs pushed to the Hugging Face Hub. I added this because Kaggle sessions can end at any time and I did not want to lose a run.	
Config	Separate `config.yaml` per language (paths, phone set, speaker count, statistics files).	

2. Data

2.1 Selection and subsampling

I looked at both datasets first (hours, clip-length percentiles, sampling rate, who the speakers are) before deciding how to subsample. Both were resampled to 22.05 kHz mono and peak-normalised. For both languages I fixed the random seed to 42, filtered the pool, and then drew a validation set of about 1 hour and a training set of about 5 hours.

	English (Svarah)	Telugu (IndicTTS Telugu)
Source	`ai4bharat/Svarah`, the only split on the Hub (`test`), used as my pool	`SPRINGLab/IndicTTS_Telugu`, `train` split
Voice(s)	Many speakers of accented English	One voice only (gender label 1). I listened to clips from both labels and kept one voice, so the model has a single speaker
Filters	1.5–15 s; characters limited to letters, digits and basic punctuation; at least 3 words; 5–25 characters per second	2–15 s; Telugu Unicode block plus basic punctuation; 5–30 characters per second; duplicate sentences removed
Split unit	Recording group (first token of the file name), so no group appears in both sets	Individual utterance; no sentence appears in both sets
Extra checks	Every validation speaker also exists in training; no validation sentence (case-insensitive) occurs in training	Assertion that utterance ids and texts do not overlap
Used after preprocessing	3,133 train / 589 val utterances 4.65 h train / 0.92 h val	2,306 train / 456 val utterances 4.27 h train / 0.86 h val

Both training sets ended a little below 5 hours. For English, 16 utterances were dropped at preprocessing because the aligner gave a phone longer than 1.5 s, which usually points to a bad alignment. [CONFIRM: why the hours after preprocessing are below the 5 h / 1 h target, e.g. silence trimmed using the alignments]

Speakers in Svarah. Svarah has no speaker id column. The file names did not give me a usable one either, so I defined a speaker as a unique combination of the demographic fields (gender, age group, mother tongue, native state and district, qualification, job category, occupation domain). That gave 115 speaker ids in the training set. It is an approximation: two real people with the same profile get merged into one id. I mention it again in the limitations.

2.2 Preprocessing

For English the first alignment run failed on about 40 transcripts. The cause was numbers: years such as 1990s, ordinals, decimals and numbers with commas, none of which MFA's dictionary can handle. I wrote a normaliser for those cases (with a fallback that reads any leftover digit as a word) and re-ran the alignment; after that every English utterance aligned. For Telugu, the mapper produced 13,195 distinct words and a dictionary with 46 of the 47 phones in use (only *rr* never occurs); the rarest phones are *jh* (4 occurrences), *ddh* (26) and *chh* (45). After training the aligner I checked the TextGrids before spending GPU time: no *spn* (unknown sound) files, median longest phone 0.21 s, maximum 1.38 s, nothing above 1.5 s.

The feature extraction follows the reference code: mel spectrograms at 22.05 kHz, phone durations from the TextGrids, F0 with PyWorld and energy from the STFT, with pitch and energy statistics (`stats.json`) computed on the training set and used for normalisation.

2.3 Training configuration

	English	Telugu
Training steps	30,000	15,000
Batch size	about 64 (worked out from 3,133/49 and 2,306/37 iterations per epoch)	
Validation	every 1,000 steps on the full validation set; best checkpoint kept by total validation loss	
Wall-clock time	5 h 47 min (about 1.44 it/s)	3 h 02 min (about 1.38 it/s)
Best checkpoint used	step 12,000 (val loss 2.608)	step 15,000 (val loss 2.066)
Optimiser, learning rate, warm-up, dropout	[FILL FROM config.yaml: optimiser, peak LR, warm-up steps, dropout]	
Training hardware	Kaggle notebook GPU [CONFIRM GPU model]; inference on a Colab T4	
Vocoder for the test audio	[CONFIRM: vocoder used to turn mels into the .wav files]	

3. Training and validation loss

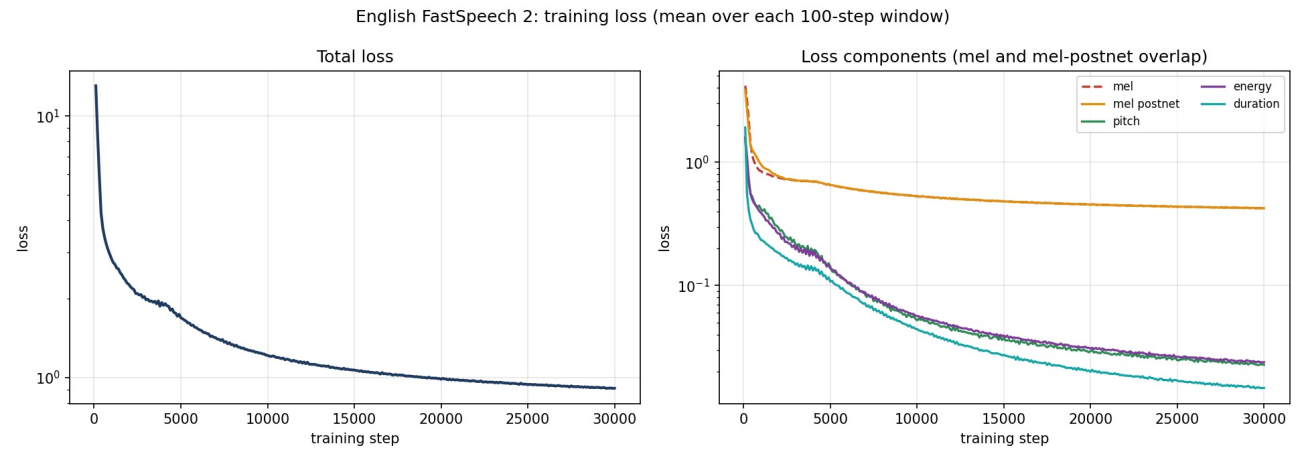


Figure 1. English training loss.

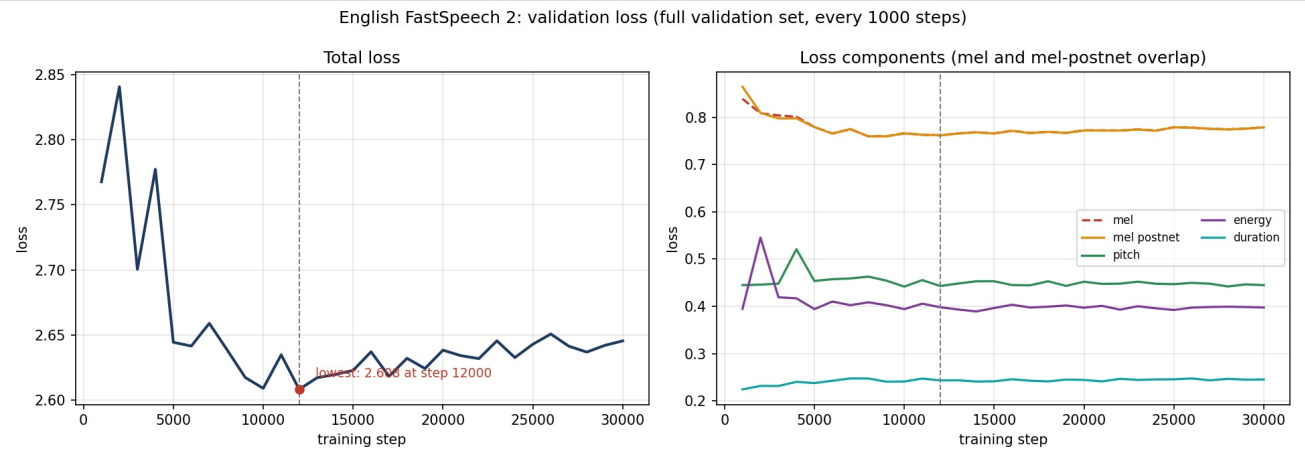


Figure 2. English validation loss. The dashed line marks the best checkpoint (step 12,000).

English

The training loss falls quickly. The total goes from about 13 at step 100 to 1.37 at step 7,600, 1.06 at step 15,100 and 0.91 at step 30,000, and it never stops going down. Validation behaves very differently. It starts at 2.77, jumps to 2.84 at step 2,000 and again to 2.78 at step 4,000 (those two bumps come from the pitch and energy terms, which are the noisy ones early on), then settles around 2.61–2.65 from step 5,000 onward. The minimum is 2.608 at step 12,000. After that it creeps up slowly to 2.645 at step 30,000, about 1.4% worse, with the mel term going from 0.762 to 0.779 while the training mel loss keeps shrinking (0.42 at the end).

So the model overfits after roughly 12,000 steps, which is not surprising with 4.65 hours of accented, multi-speaker speech. The increase is small and there is no blow-up, but it is the reason I synthesised the test sentences with the step-12,000 checkpoint and not the final one. The gap for pitch and energy is much bigger than for mel: at the end the training pitch loss is 0.023 and the validation pitch loss is 0.445, so those two predictors memorise the training utterances. The duration loss is also flat but high (about 0.24 on validation).

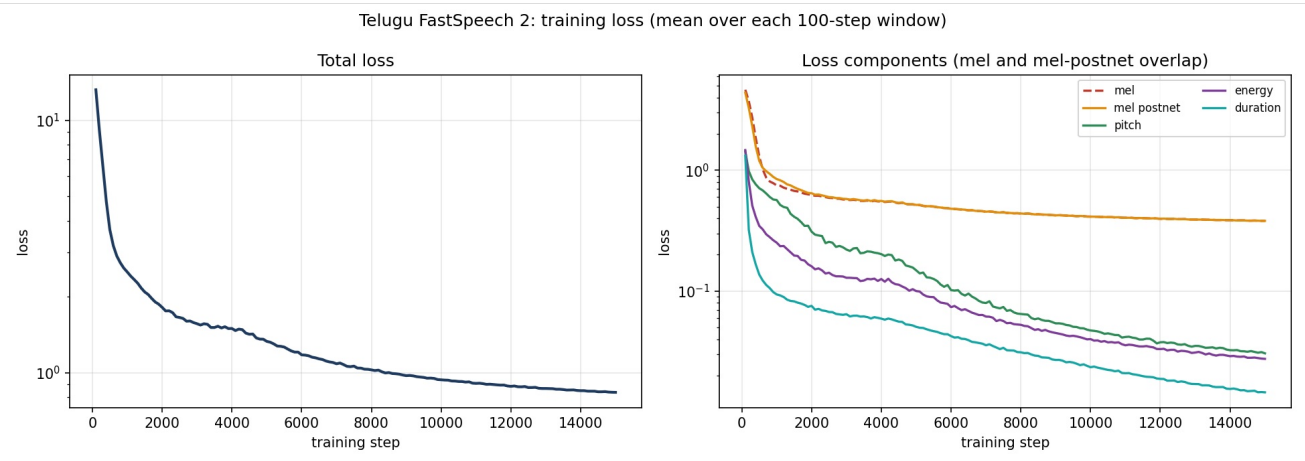


Figure 3. Telugu training loss.

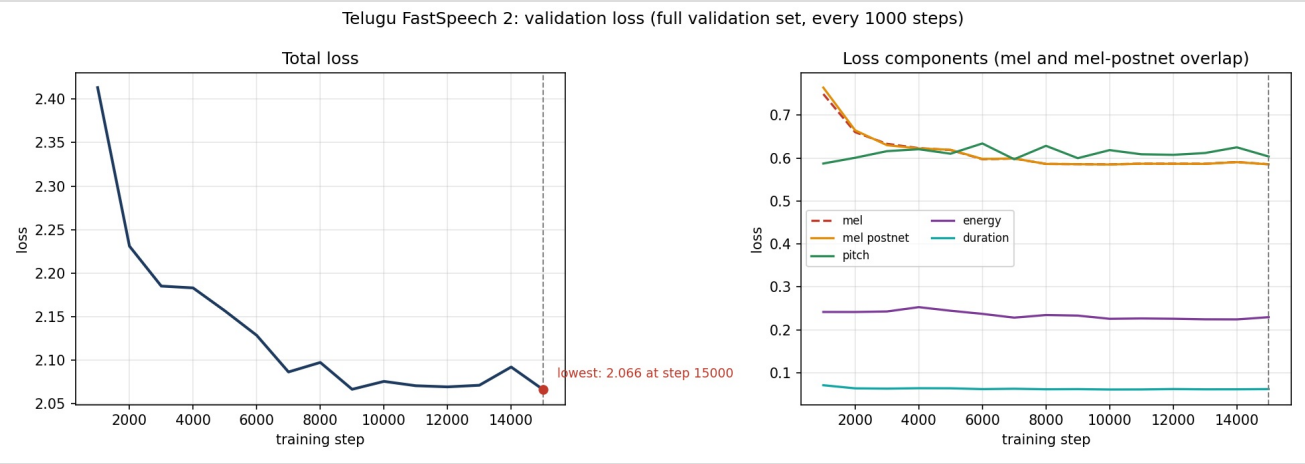


Figure 4. Telugu validation loss.

Telugu

Telugu looks cleaner. Validation loss goes from 2.41 at step 1,000 to 2.19 at step 3,000 and 2.07 at step 9,000, then stays between 2.066 and 2.092 until step 15,000, which is also where the best checkpoint is. I stopped at 15,000 steps, so I cannot say what would have happened later; the plateau of the last six thousand steps suggests it is close to what this amount of data allows, and the English run suggests that continuing would start to overfit. The mel validation loss has been flat at 0.586 since step 8,000. The duration loss is very small (0.061, against 0.24 for English), which fits the alignments being good: one speaker and an aligner trained on exactly this audio.

One thing I noticed is that the validation pitch loss never improved in Telugu. It was 0.587 at step 1,000 and ends at 0.604, moving between 0.60 and 0.63 in between, while the training pitch loss drops to 0.031. The pitch predictor learns the training utterances but does not generalise much further than its first guess. This matches what I see in the F0 plots (section 5), where the predicted Telugu contours are wide and sometimes extreme.

4. Generated speech: pronunciation and quality per test sentence

All audio was generated with the best checkpoint of each language (English step 12,000, Telugu step 15,000), English with speaker id 0. The five sentences per language are E1, E2a, E2b, E3, E4 and T1, T2a, T2b, T3, T4. In the phone sequences below I use what the model was given as input, so a mistake in the front end shows up directly in the audio. [ADD BY EAR: for each sentence, one line on naturalness, artefacts, robotic or buzzy sound, clipped words — I wrote this section from the phone sequences and the plots; it needs a listening pass before submission]

English

> **E1** – The antidisestablishmentarianism movement gained considerable momentum during the parliamentary proceedings.

The long word is converted into 28 phones with a sensible stress pattern: secondary stress on the first and the middle syllable (AE2) and primary stress on the syllable before the ending (EH1 in “-tar-”). The rest of the sentence uses ordinary dictionary words, so I expected no front-end problems, and the phone sequences have none.

> **E2a / E2b** – a) The orchestra's conduct during the ceremony was praised by the audience. b) The researchers will conduct an experiment to evaluate the proposed method.

This is the homograph test. In (a), *conduct* is a noun and gets stress on the first syllable (K AA1 N D AH0 K T). In (b) it is a verb and gets stress on the second syllable (K AA0 N D AH1 K T). So the front end resolved both correctly; my guess is that this comes from the part-of-speech based homograph handling of `g2p_en`, but I did not test that separately. Because the pronunciation is decided before FastSpeech 2 sees the text, the model itself has no part in getting this right.

> **E3** – The rhyth m of the orchestra changed unexpectedly.

I kept the text exactly as given in the assignment, including the stray space in “rhyth m”. The result is a pronunciation failure: “rhyth” becomes R AY1 DH (like “rythe”) and the loose “m” is read as the letter name, EH1 M, so the listener gets something like “rye-th em”. As a diagnostic I also generated the sentence with the space removed (`E3_fixed`): “rhythm” becomes R IH1 DH AH0 M, which is correct. So the problem is entirely the broken input word, not the model.

> **E4** – The meeting will start at 10:45 AM on 12 November 2026, so andaroo should arrive on time.

The normaliser turns the sentence into “ten forty five A M on twelfth of November twenty twenty six”, which is the

natural way to read it; “AM” is spelled as two letters and “12” becomes the ordinal “twelfth”. *andaroo* is not in the dictionary (it is Telugu for “everyone”), so G2P guessed AE0 N D AA1 R OW0, i.e. “an-DAA-roh”. That is a plausible reading of the spelling, but it is a guess made by the G2P model and I did not check it against how the word is actually said. The comma is rendered as a pause.

Telugu

The sentence list in the assignment PDF has broken spacing and some detached diacritics in the Telugu text (it looks like a copy-paste artefact: words glued together or split inside a word). I repaired the spacing for T1, T2a, T2b and T4. For T3 I synthesised the text *as it appears in the PDF* (“అడ విలో౯”, which should be అడవిలో) and also the repaired version (`T3_joined`), so both are in the results folder.

> **T1** – న్యూరోఫార్మకోలాజికల్ పరిశోధనలో నూతన ఔషధాల ప్రభావాన్ని విశ్లేషించారు.

The first word is a long English loanword (“neuropharmacological”) written in Telugu script. The mapper turns it into 20 phones, including *ph*, which is rare in the training set (155 occurrences, like *au* in ఔషధాల with 164). The mapping itself is consistent with how the word is written. This is the sentence where I expected the weakest result, because the model has seen few examples of these sounds.

> **T2a / T2b** – a) తాడుపై అతని పట్టు చాలా బలంగా ఉంది. b) ఆమె పట్టు చీరను ధరించి వేడుకకు వచ్చింది.

పట్టు means “grip” in (a) and “silk” in (b). It is written and pronounced the same in both (p a tt tt u), so unlike the English homograph test there is nothing for the front end to decide, and any difference between the two can only appear in the prosody. Nasal handling looks correct in both sentences: the anusvara becomes the nasal that matches the next consonant (బలంగా → b a l a **ng** g aa, ఉంది → u **n** d i, ధరించి → dh a r i **ny** ch i).

> **T3** – పులి అడ విలో నిశ్శబ్దంగా నడిచింది.

With the text as given, the word అడవిలో is split into అడ + విలో (a dd a / v i l oo). The phones are the same as the joined version, only the word boundary differs, and I could not see any acoustic consequence of it (details in section 5). The long consonant శ్శ in నిశ్శబ్దంగా is mapped to a doubled sh (*sh sh*).

> **T4** – ఈ సమావేశం 2026 నవంబర్ 12వ తేదీ ఉదయం 10:45 గంటలకు ప్రారంభమవుతుంది, సమయానికి సిద్ధంగా ఉంచండి.

The numerals are expanded into words before synthesis: 2026 → రెండు వేల ఇరవై ఆరు, 12వ → పన్నెండవ, 10:45 → పది నలభై ఐదు. That is correct for the date and the time. The comma becomes a pause. This sentence has 120 phones and is the longest test input in either language.

5. Variance adaptor: duration, pitch and energy

For each test sentence I plotted the predicted duration, F0 and energy per phone (files in `Results/<Language>/variance_adaptor_plots/`, with the underlying numbers in the `data/` folder next to them). A frame is 256 samples at 22.05 kHz, about 11.6 ms, so durations are multiples of that. No sentence has a zero-length phone, and no sentence has a pause other than at a comma. The two figures below show the longest sentence of each language; the rest are in the repository.

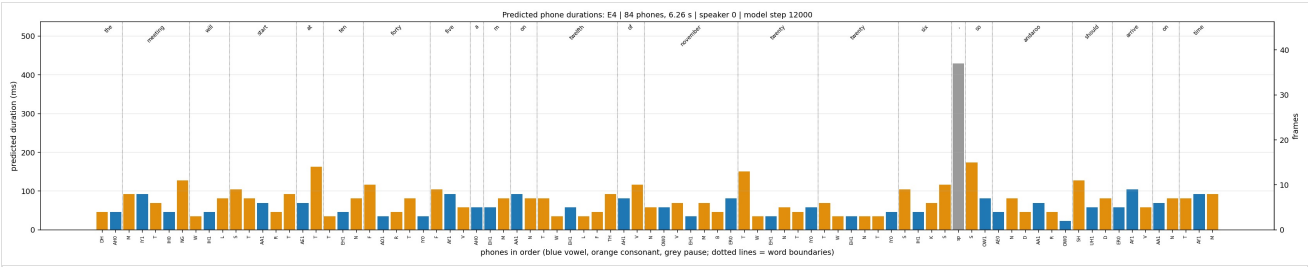


Figure 5a. E4, predicted duration per phone (the shaded block is the pause at the comma).

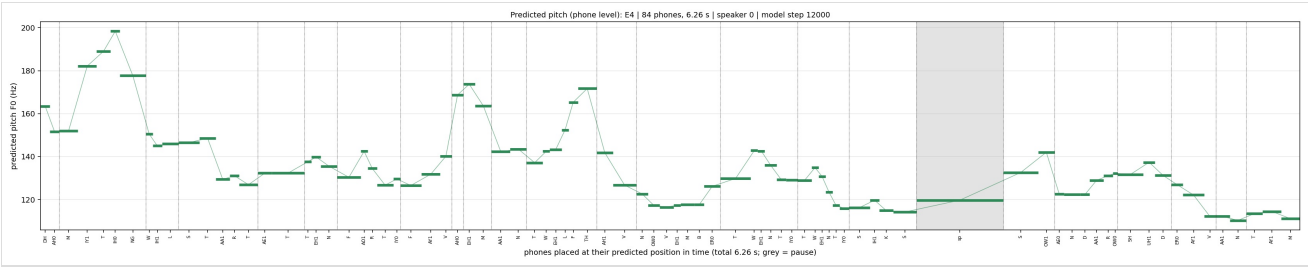


Figure 5b. E4, predicted F0.

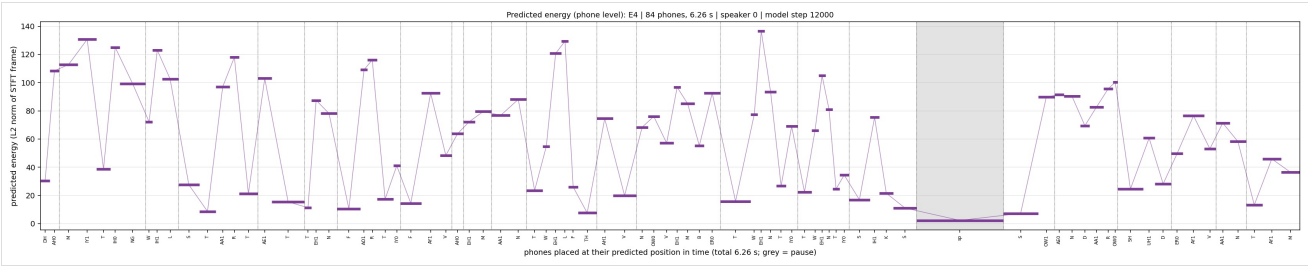


Figure 5c. E4, predicted energy.

ID	Phones	Length (s)	Mean phone (ms)	Pause (ms)	F0 first → last (Hz)	F0 min–max (Hz)	Energy peak on
E1	87	5.36	61.7	–	135 → 113	105–169	considerable
E2a	51	3.79	74.2	–	147 → 97	97–153	orchestra's
E2b	56	3.66	65.3	–	173 → 111	111–173	proposed
E3	37	2.57	69.3	–	140 → 115	115–156	the
E4	84	6.26	70.2	430	164 → 111	110–198	twenty
T1	66	4.64	70.4	–	134 → 111	109–280	న్యూరోఫార్మకోలాజికల్
T2a	31	2.14	68.9	–	169 → 128	126–228	చాలా
T2b	37	2.23	60.2	–	155 → 106	106–253	చీరను
T3	31	1.77	56.9	–	194 → 120	120–272	విలో
T4	120	7.31	59.0	290	195 → 114	111–257	పన్నెండవ

English

Common pattern. In all five sentences F0 starts high and ends at its lowest or close to it (147→97 Hz in E2a, 173→111 Hz in E2b), so the model has learned the falling pitch of a plain statement. Phrase-final lengthening is also visible: the last syllable of *unexpectedly* in E3 (IY0, 255 ms) is the longest phone of that sentence. Mean phone length is between 62 and 74 ms. Energy goes from about 6–11 at its lowest to 119–147 at its peak.

E1. There is no pause in the sentence. *antidisestablishmentarianism* takes 1.72 s, about a third of the whole sentence, and its average of 61 ms per phone is the same as the sentence average, so the duration predictor simply scales with the number of phones and does not compress the long word. The sentence's F0 peak (169 Hz) is inside this word, rising from 120 Hz, and the lowest F0 (105 Hz) is inside *considerable*, which is also where energy peaks (119) — not in the long word. The final word *proceedings* falls from 127 to 113 Hz.

E2a and E2b. The same word *conduct* gets different prosody. As a noun in (a) it takes 499 ms and F0 inside the word ranges from 114 to 153 Hz, the F0 peak of the sentence. As a verb in (b) it takes 453 ms and stays between 133 and 149 Hz, lower than the start of the sentence. In (b) the F0 peak (173 Hz) is on the very first word *the*, which tells me the pitch predictor is mostly applying a high start plus gradual decline rather than placing accents on meaningful words. (a) is also the slowest sentence (74 ms per phone), with some long fricatives (an /s/ of 186 ms), and F0 falls by about 50 Hz over the sentence.

E3 and E3_fixed. The split word “rhyth m” costs 197 + 186 = 383 ms against 337 ms for the correct *rhythm*, and the loose *m* gets its own pitch peak (156 Hz) as if it were a stressed word. In the fixed version the F0 peak moves to *of* (169 Hz) and the energy peak (147) moves to *rhythm*, instead of sitting on *the* (129) as in the broken version. So the plots show the same thing as the audio: the broken input makes the predictors treat a stray letter as a word.

E4. The longest English sentence (6.26 s) and the only one with a pause: 430 ms at the comma. F0 starts at 164 Hz and peaks at 198 Hz on *meeting*, the highest of all English test sentences, then moves up again on “A M” (164–174 Hz) and *twelfth* (up to 172 Hz). The letters and the date get raised pitch, which seems a natural pattern for reading out a time and a date. *twenty twenty six* has the energy peak of the sentence (136). After the comma F0 resets to 133–142 Hz on *so* and falls to 111 Hz at the end, so the model realised two intonation phrases.

Telugu

Common pattern. The contour shape is the same as in English (high start, low end: 195→114 Hz in T4, 194→120 Hz in T3), but the pitch values are very different: mean F0 per sentence is 183–193 Hz with a standard deviation of 26–37 Hz, against 131–139 Hz and 11–19 Hz in English. Phones are slightly shorter on average too (63 ms against 68 ms in English, averaged over the five sentences), although the two phone inventories are not directly comparable. [CONFIRM: a mean F0 of about 185 Hz is in the usual range of a female voice; I selected gender label 1 as “male” — check the label mapping of the dataset and the audio]

T1. The first word takes 1.51 s (20 phones, 75 ms per phone) and F0 climbs inside it from 134 to 279 Hz, a 145 Hz rise within one word, and the 279 Hz peak is the highest F0 prediction in the whole test set. The energy peak (131) is also in this word. I think this is the pitch predictor over-reacting to a word made of rare phones (it matches the validation pitch loss that never improved). The remaining words descend steadily: 182–242 Hz in పరిశోధనలో down to 109–181 Hz in the last word.

T2a and T2b. పట్టు has the same phones in both sentences and the model realises it almost identically: 302 ms with F0 185–228 Hz in (a), 290 ms with 178–227 Hz in (b). That is what I would expect from a text-only front end, since nothing in the input tells the two meanings apart. The differences are in the sentence contour. (a) peaks on పట్టు (228 Hz) and has its energy peak on the intensifier చాలా ("very"), which makes sense. (b) peaks late, on వేడుకకు at 253 Hz, and has the lowest energy peak of all Telugu sentences (104, on చీరను).

T3 and T3_joined. All summary statistics are identical for the two versions: 31 phones, 1.765 s, F0 range 120–272 Hz, no pause. So the extra space in అడ విలో has no effect at all; the whitespace creates a word boundary in the text but not a pause in the output. This is the shortest and the fastest sentence (56.9 ms per phone). F0 peaks at 272 Hz on the first word పులి and the longest word is నిశ్శబ్దంగా (11 phones, 626 ms).

T4. 7.31 s with one pause of 290 ms at the comma. F0 starts at 195 Hz, peaks at 257 Hz on సమావేశం, falls to 124 Hz at the end of the first clause, jumps back to 164–226 Hz after the comma and ends at 111–149 Hz on ఉంచండి. As in English, that is two intonation phrases. The longest word ప్రారంభమవుతుంది (17 phones) takes 859 ms, which is about 50 ms per phone, faster than the sentence average of 59 ms, so here the duration predictor does compress a long word. The spoken numerals (రెండు వేల ఇరవై ఆరు and so on) get F0 between 150 and 230 Hz.

6. English compared with Telugu

The two models behave differently mainly because the two datasets are so different. Telugu is one studio voice with a rule-based phone set and an aligner trained on the same audio. English is accented speech from many speakers, labelled with phones from a pretrained US-English aligner. This shows in the numbers. The Telugu duration validation loss is 0.061, the English one 0.24; Telugu validation loss stops improving but does not rise, while English starts to rise after step 12,000. Total losses of the two languages are not directly comparable, since the data and the normalised pitch and energy are different.

The prosody the models predict is also different. English contours are narrow (F0 standard deviation 11–19 Hz around a mean of about 135 Hz), Telugu contours are wide (26–37 Hz around about 185 Hz) and include extreme single-word jumps like the 134→279 Hz in T1. It could be partly real (a more expressive speaker), partly a pitch-tracking problem in the training data (octave errors in PyWorld push values up) and partly the predictor over-fitting; I did not separate these causes.

Limitations and failures

The clearest failure is E3: the model is fine, but the input "rhyth m" is read as two words and the audio is wrong. The same thing could happen to Telugu text with broken spacing, although for T3 the broken input did not change the output. The 115 "speakers" of the English model are demographic profiles and not real people, so the speaker embedding is partly meaningless and speaker 0 is only the profile with the most data. English overfits after 12k steps and the pitch and energy predictors memorise the training set in both languages. Out-of-vocabulary words (*andaroo*) depend on a G2P guess, and the Telugu front end is rule-based, so loanwords with rare phones (T1) are the weakest case. Numbers and dates are handled by hand-written rules and I only tested them on the sentences of this assignment. Finally the amount of data (4.3–4.7 hours) is small for FastSpeech 2, and I did not run a listening test or an objective measure such as MCD, so quality statements beyond the plots are subjective.

What I would try next

A proper early-stopping rule and a stronger regulariser (more dropout, weight decay, or a smaller model) for English. A text normaliser with unit tests that also removes stray spaces inside words. A real speaker id for English, for example by using only a few large speakers or clustering the audio. Smoothing F0 and removing octave jumps from the training pitch, which could make the Telugu pitch predictor more stable. A shared phone inventory for both languages so the model can use all 9 hours. And a listening test with a few people, plus MCD and word error rate through an ASR model, to get numbers for speech quality.

7. Reproducibility

Software: Python 3, PyTorch, PyWorld, `tgt`, `g2p_en`, `num2words`, `datasets`, `huggingface_hub` (see `requirements.txt`), and Montreal Forced Aligner 3.x installed through conda-forge. Each language folder has the three scripts the assignment asks for, `preprocess.py`, `train.py` and `inference.py`, plus its own `config.yaml` (run settings: paths, run name, number of steps, inference

options). They are my notebook pipeline as scripts and call the FastSpeech 2 core code that sits in `<Language>/fastspeech2/` together with its model `config.yaml`; at run time they rewrite the Kaggle paths in that file to local folders. Commands, from the repository root:

```
# once: put the core code in English/fastspeech2 and Telugu/fastspeech2
python tools/merge_code_zips.py English_code.zip Telugu_code.zip
export HF_TOKEN=hf_xxx

cd English        # then the same in Telugu/
python preprocess.py   # data selection (seed 42) -> alignment -> features
                       # (--from-hf reuses the prepared data and alignments)
python train.py        # 30,000 steps (Telugu 15,000), resumes automatically
python inference.py    # test sentences, audio and variance-adaptor plots -> ../Results/English
```

Checkpoints: `English/checkpoints/best.pth.tar` (step 12,000) and `Telugu/checkpoints/best.pth.tar` (step 15,000), about 142 MB each, so they are stored with Git LFS and also available at [ADD DOWNLOAD LINK]. Seed 42 is used for the data split.